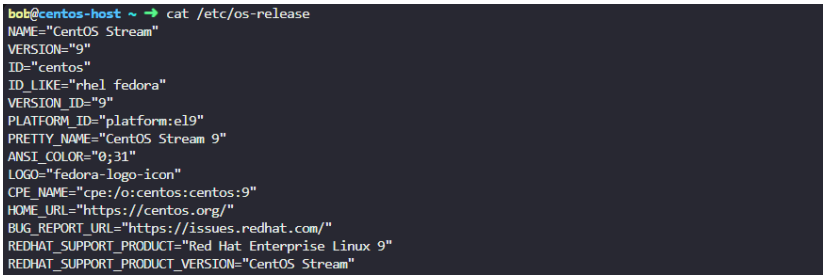
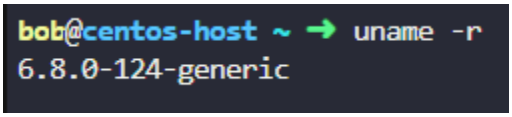
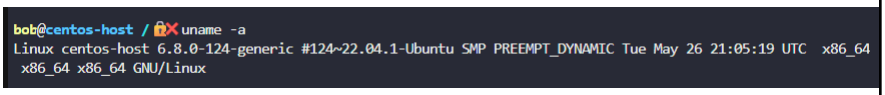
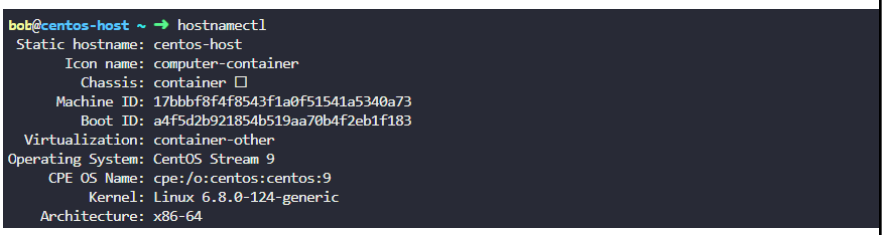
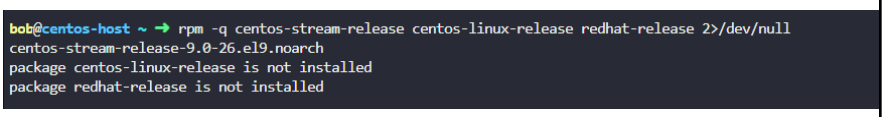
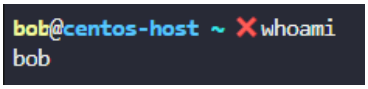
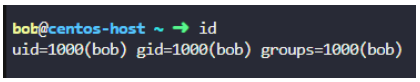
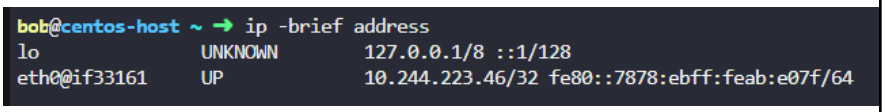
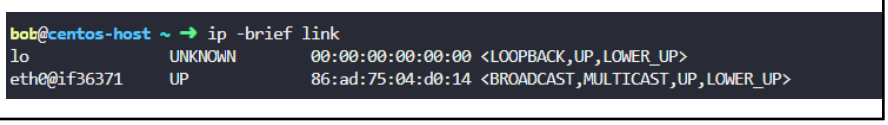
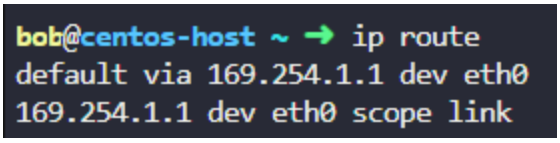
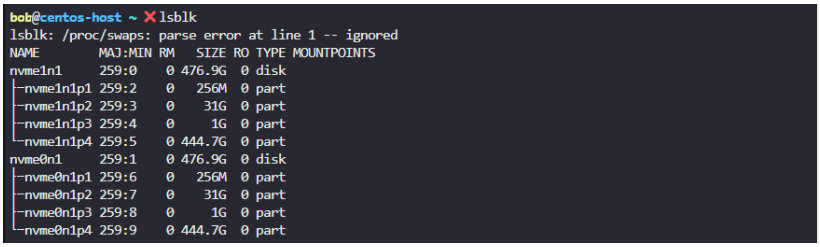
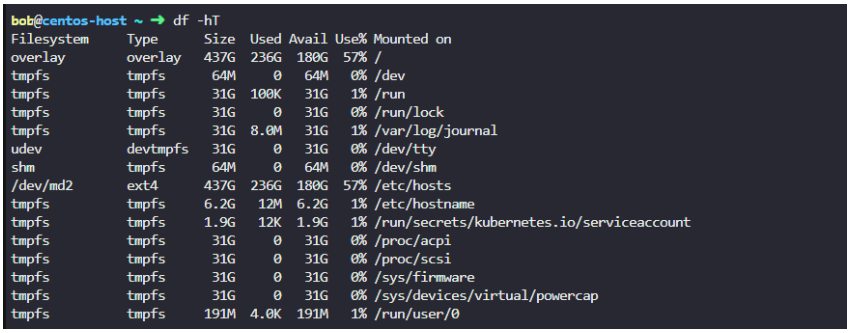
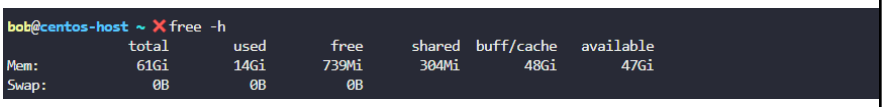
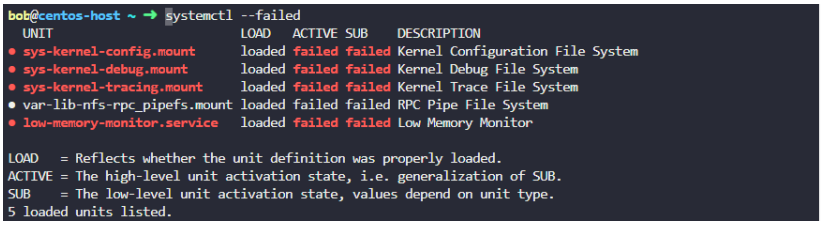
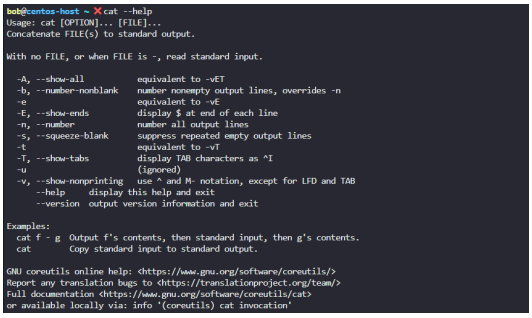
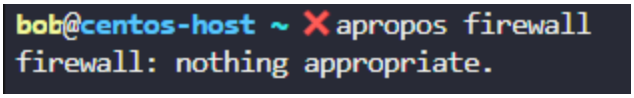
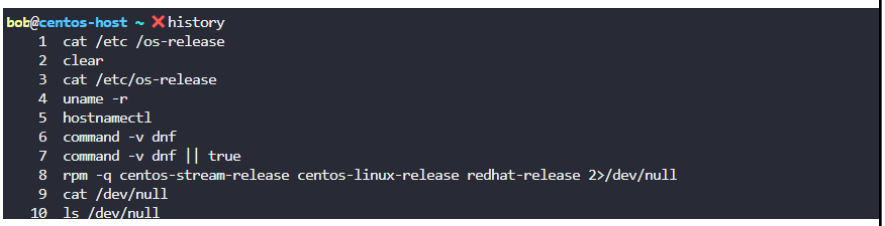
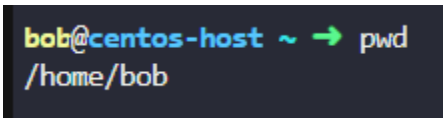
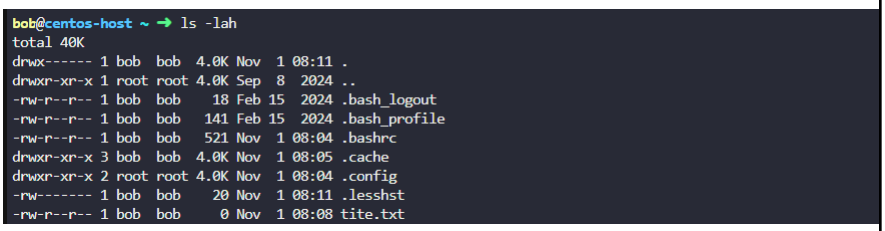
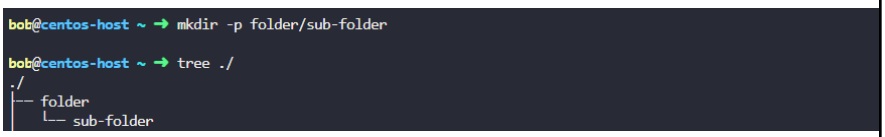
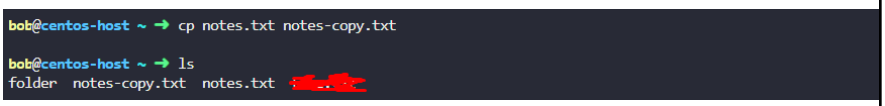
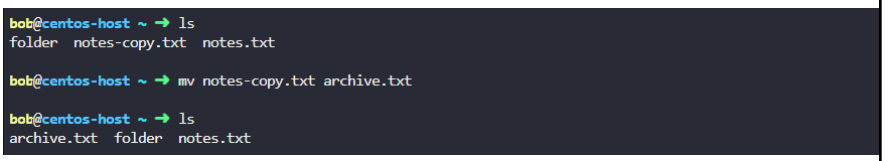
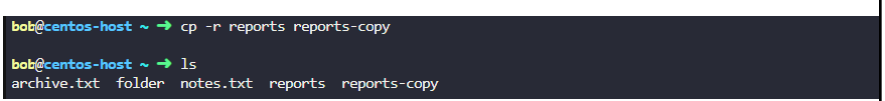
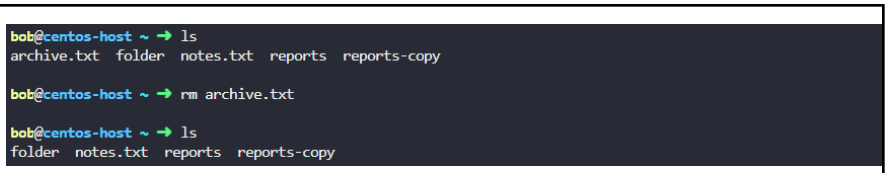
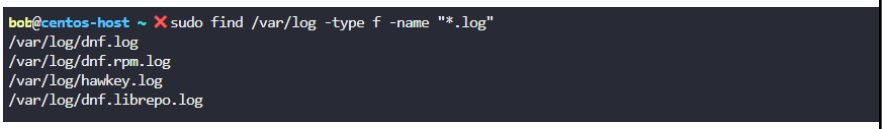
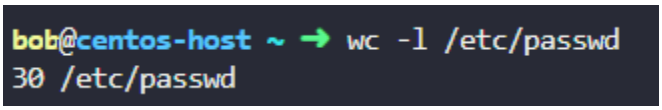
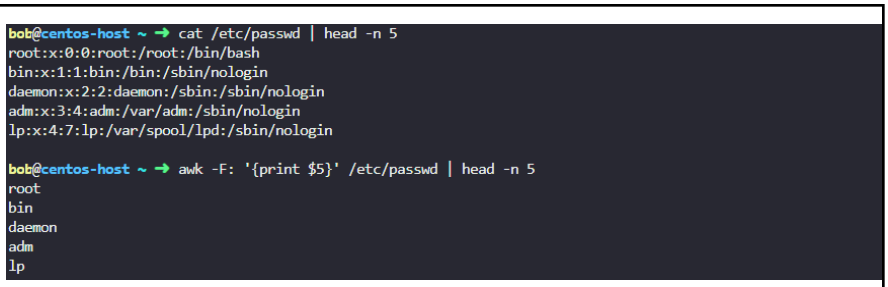
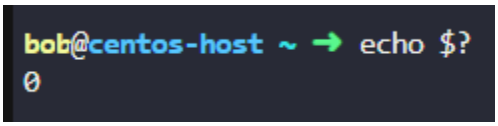
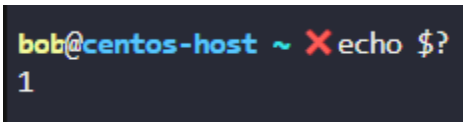
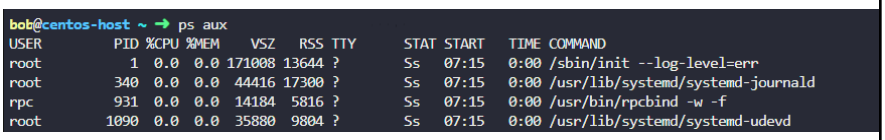
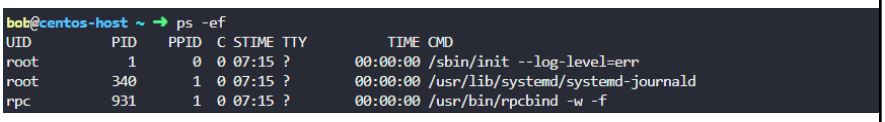
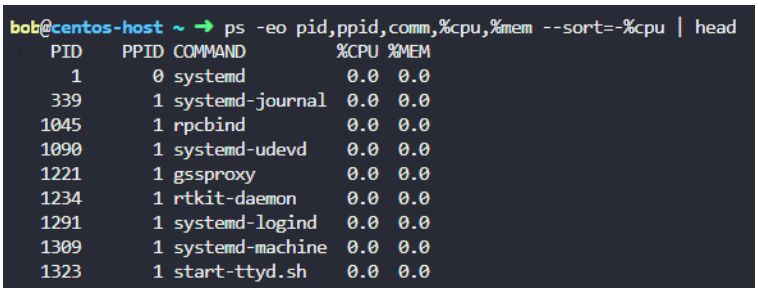
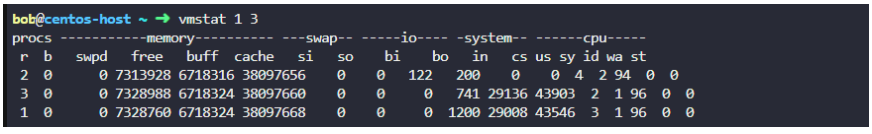
Remember packages: NetworkManager, httpd, firewalld, tree, xfsprogs, bind-utils, cronie, nmap
Do: sudo setenforce 0 (SELinux Disable) | check sestatus

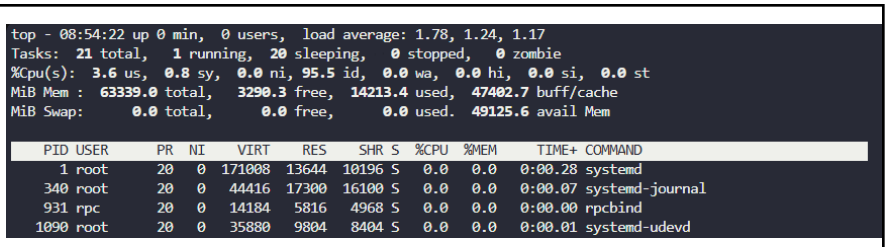
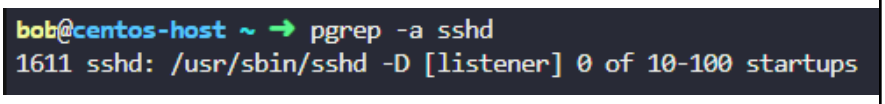
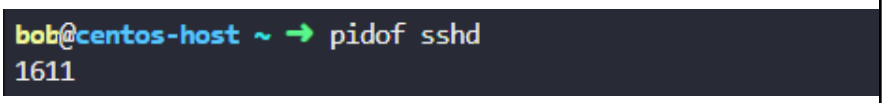
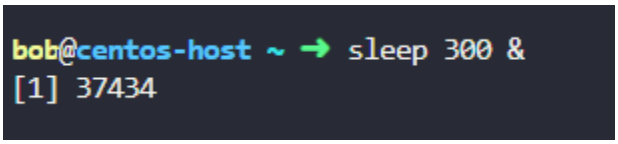
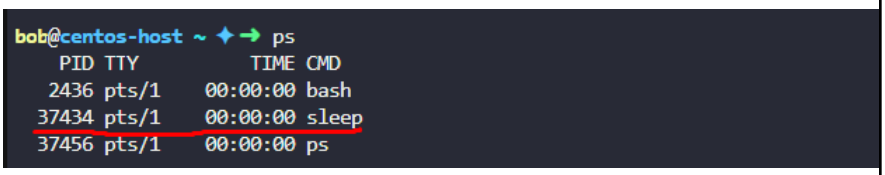
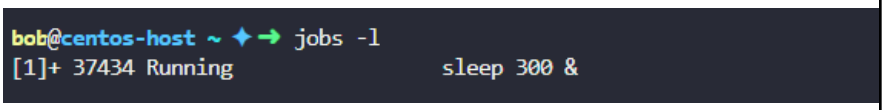
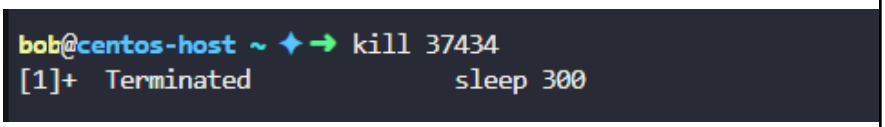
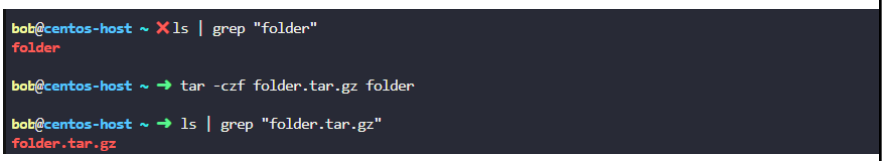
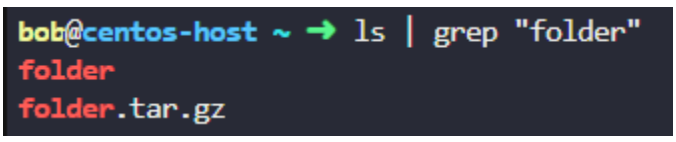
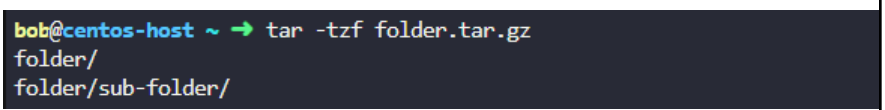
cat /etc/os-release	
uname -r	
uname -a	
hostnamectl	
command -v dnf	
rpm -q centos-stream-release centos-linux-release redhat-release 2>/dev/null	
whoami	
id	
ip -brief address	
ip -brief link	

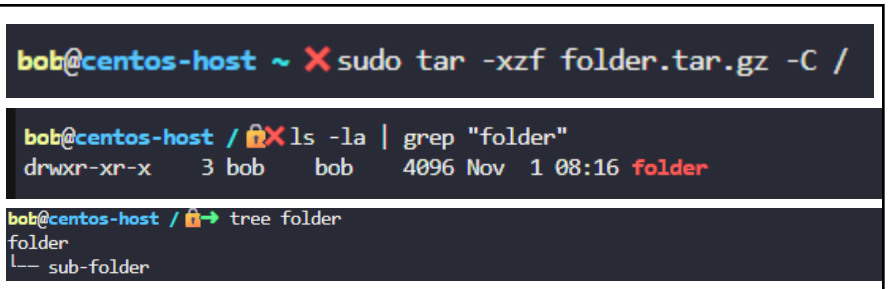
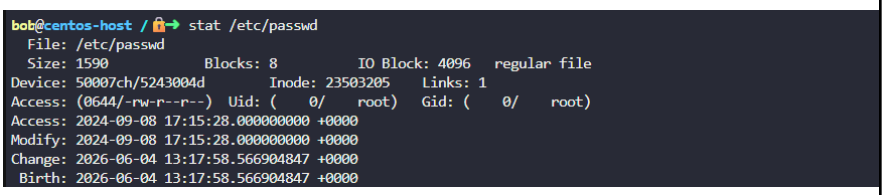
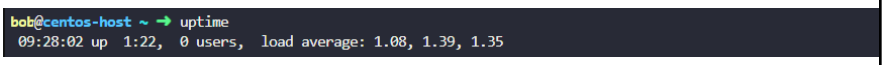
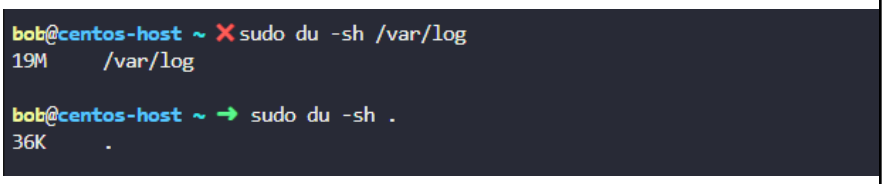
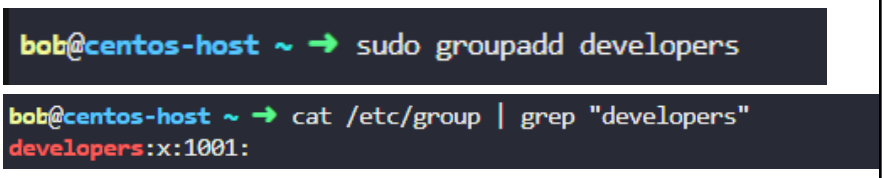
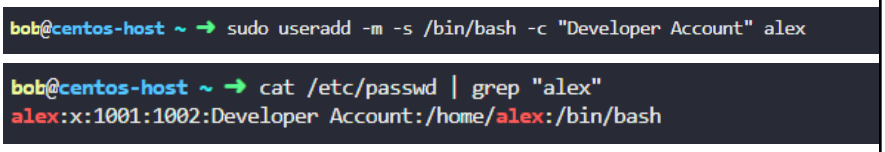
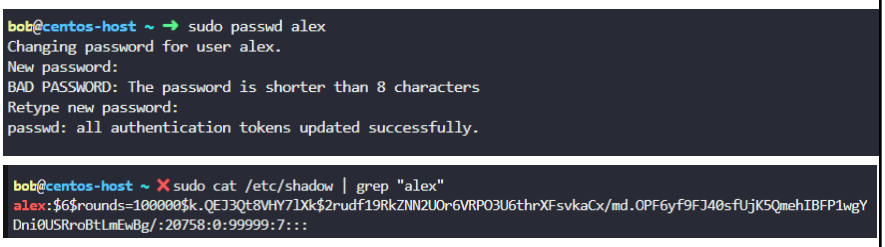
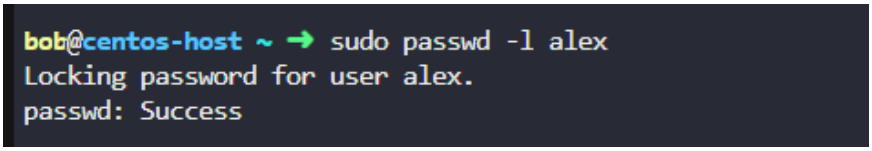
ip route	
ip route get 1.1.1.1	
lsblk	
df -hT	
free -h	
systemctl --failed	
man grep	MANUAL OF GREP
cat --help	

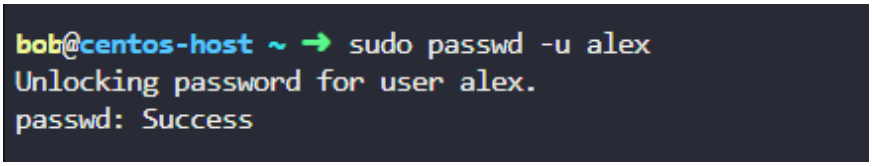
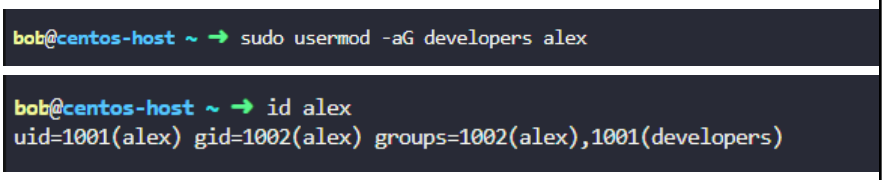
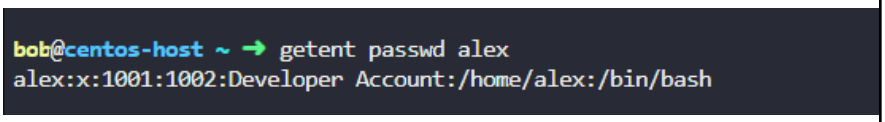
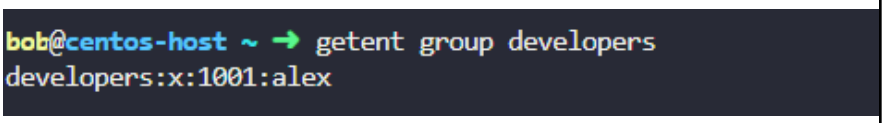
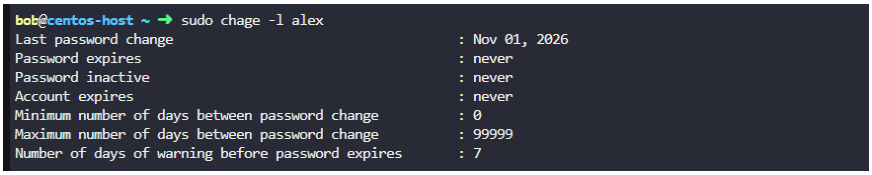
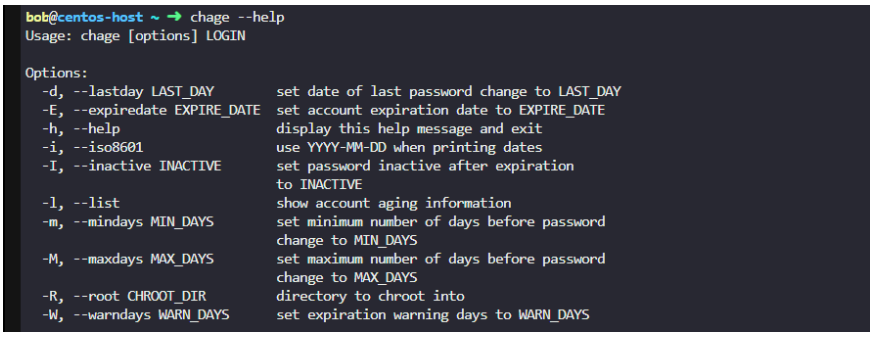
apropos firewall	
type -a grep	
history	
pwd	
ls -lah	
mkdir -p	
touch notes.txt	
cp notes.txt notes-copy.txt	
mv notes-copy.txt archive.txt	
cp -r reports reports-copy	

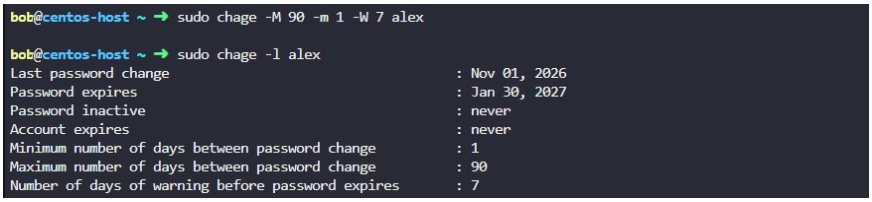
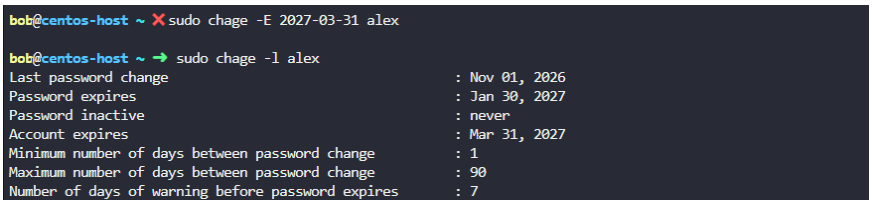
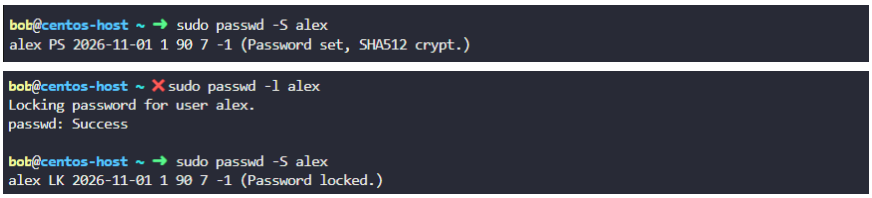
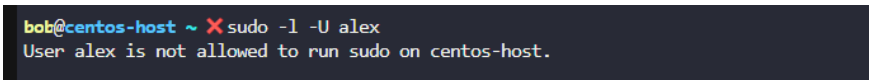
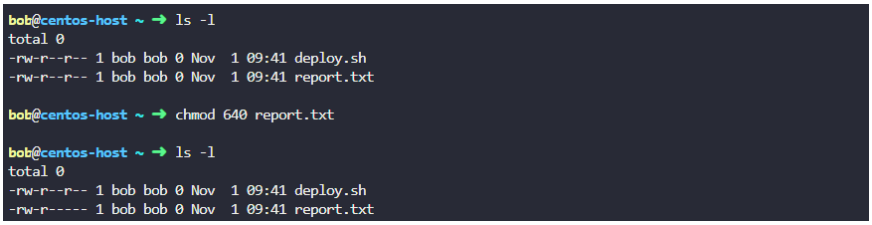
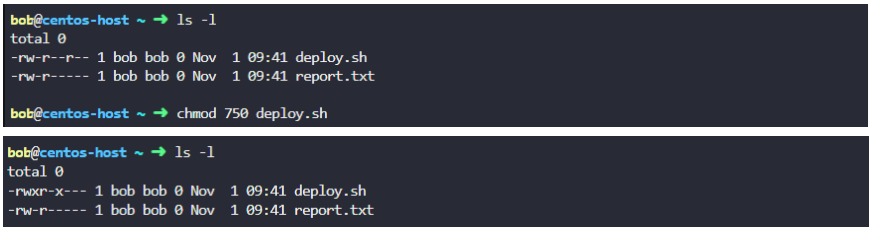
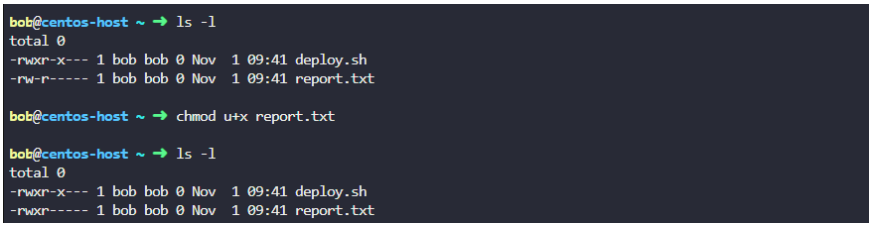
rm archive.txt	
rm -r reports-copy	
cat /etc/hostname	
head -n 5 /etc/passwd	
less /var/log/messages	
tail -f /var/log/messages	
grep -n “ bob” /etc/passwd	
grep -i “bob” /etc/passwd	
find /var/log -type f -name "*.log"	
wc -l /etc/passwd	
sort /etc/passwd | uniq -c	
cut -d: -f1 /etc/passwd	

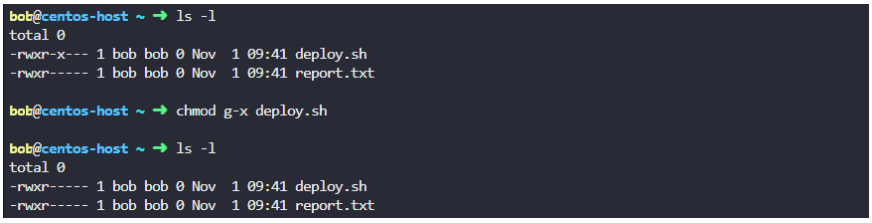
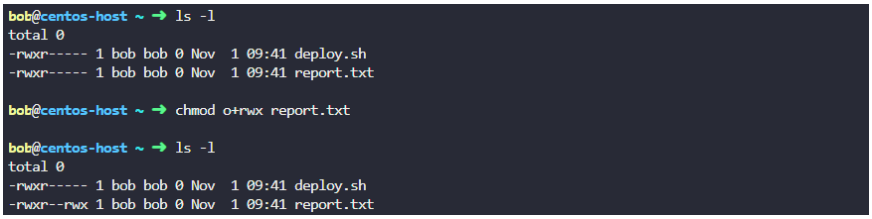
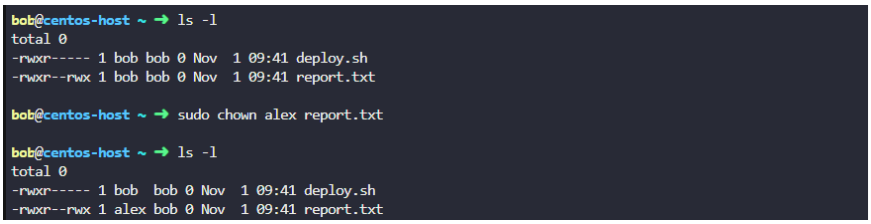
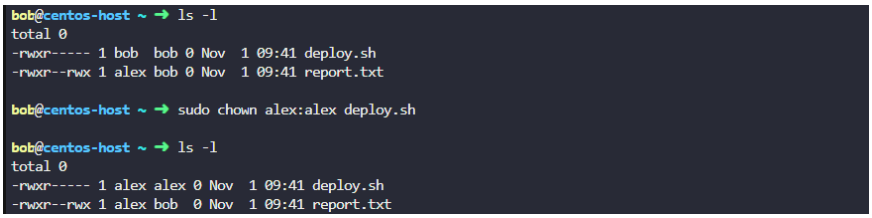
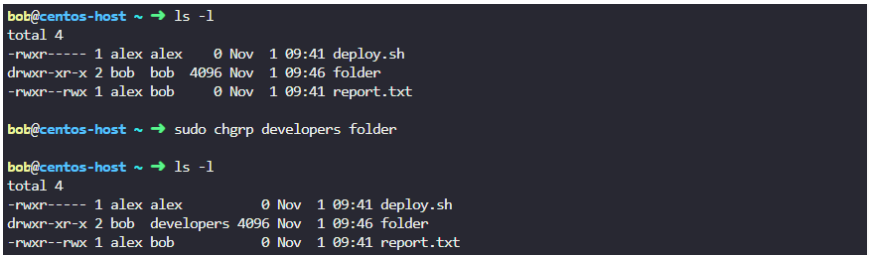
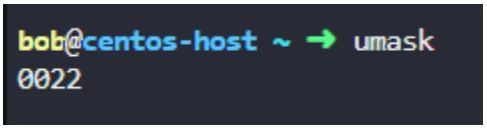
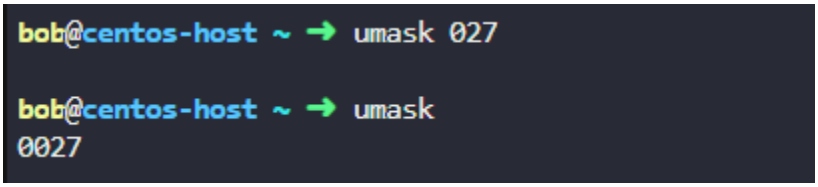
awk -F: '{print $5}' /etc/passwd | head -n 5	
‘command’ > file.txt	overwrite
‘command’ >> file.txt	appends
‘command’ 2> errors.txt	error outputs
2> errors.txt >&1	stdout + stderr
command1 | command2	piping
echo $? 0 if success 1 if failed/error	 
ps aux	
ps -ef	
ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | head	
vmstat 1 3 (loop 1 to 3) vmstat 2 (run every 2sec)	

top	
pgrep -a sshd	
pidof sshd	
sleep 300 &	
ps	
jobs -l	
kill 37434	
tar -czf folder.tar.gz folder	 
tar -tzf folder.tar.gz	

tar -xzf folder.tar.gz -C /	
stat /etc/passwd	
uptime	
du -sh /var/log	
groupadd developers	
sudo useradd -m -s /bin/bash -c "Developer Account" alex	
passwd alex	
passwd -l alex	

passwd -u alex	
sudo usermod -aG developers alex	
getent passwd alex	
getent group developers	
sudo userdel alex sudo userdel -r alex	sudo userdel alex sudo userdel -r alex Command | Deletes User Account? | Deletes Home Directory (/home/alex)? | Deletes Mail Spool? sudo userdel alex | Yes | ❌ No | ❌ No sudo userdel -r alex | Yes | Yes | Yes
chage -l alex	 

chage -M 90 -m 1 -W 7 alex	
chage -E 2027-03-31 alex	
sudo passwd -S alex	
sudo -l -U alex	
chmod 640 report.txt	
chmod 750 deploy.sh	
chmod u+x report.txt	

chmod g-x deploy.sh	
chmod o+rwx report.txt	
chown alex report.txt	
chown alex:alex deploy.sh	
chgrp developers folder	
umask	 files 666 - 0022 = 644 | folder 777 - 0022 = 755
umask 027	

<table>
<tr>
<td>(SUID) Set User ID
allows user to run as root
chmod 4755 file.txt
or
chmod u+s file.txt

if capital S, meaning there is no “x” execute existing on the file

• do chmod u+x</td>
<td>

```
bob@centos-host ~/folder ➜ ls -l
total 0
-rw-r----- 1 bob bob 0 Nov  1 09:50 file.txt

bob@centos-host ~/folder ➜ sudo chmod 4755 file.txt

bob@centos-host ~/folder ➜ ls -l
total 0
-rwsr-xr-x 1 bob bob 0 Nov  1 09:50 file.txt
```

```
bob@centos-host ~/folder ➜ ls -l
total 0
-rwsr-xr-x 1 bob bob 0 Nov  1 09:50 file.txt
-rwSr----- 1 bob bob 0 Nov  1 09:51 file2.txt

bob@centos-host ~/folder ➜ chmod u+x file2.txt

bob@centos-host ~/folder ➜ ls -l
total 0
-rwsr-xr-x 1 bob bob 0 Nov  1 09:50 file.txt
-rwsr----- 1 bob bob 0 Nov  1 09:51 file2.txt
```

</td>
</tr>
<tr>
<td>(GUID) Set Group ID
allows group to run as root
chmod 2755 file.txt
or
chmod g+s file.txt

if capital S, meaning there is no “x” execute existing on the file

• do chmod g+x</td>
<td>

```
bob@centos-host ~/folder ➜ ls -l
total 0
-rwsr-xr-x 1 bob bob 0 Nov  1 09:50 file.txt
-rwsr----- 1 bob bob 0 Nov  1 09:51 file2.txt

bob@centos-host ~/folder ➜ chmod 2755 file.txt

bob@centos-host ~/folder ➜ ls -l
total 0
-rwxr-sr-x 1 bob bob 0 Nov  1 09:50 file.txt
-rwsr----- 1 bob bob 0 Nov  1 09:51 file2.txt
```

</td>
</tr>
<tr>
<td>Sticky
owned by the user can only delete/rename file

chmod 1777 file.txt
or
chmod o+t file.txt
if capital T, meaning there is no “x” execute existing on the file

• do chmod o+t</td>
<td>

```
bob@centos-host ~/folder ➜ ls -l
total 0
-rwxr-sr-x 1 bob bob 0 Nov  1 09:50 file.txt
-rwsr----- 1 bob bob 0 Nov  1 09:51 file2.txt

bob@centos-host ~/folder ➜ chmod 1777 file.txt

bob@centos-host ~/folder ➜ ls -l
total 0
-rwxrwxrwt 1 bob bob 0 Nov  1 09:50 file.txt
-rwsr----- 1 bob bob 0 Nov  1 09:51 file2.txt
```

</td>
</tr>
<tr>
<td>getfacl</td>
<td>

```
bob@centos-host ~/folder ➜ getfacl .
# file: .
# owner: bob
# group: developers
user::rwx
group::r-x
other::r-x
```

</td>
</tr>
<tr>
<td>setfacl -m u:alex:rwx .</td>
<td>

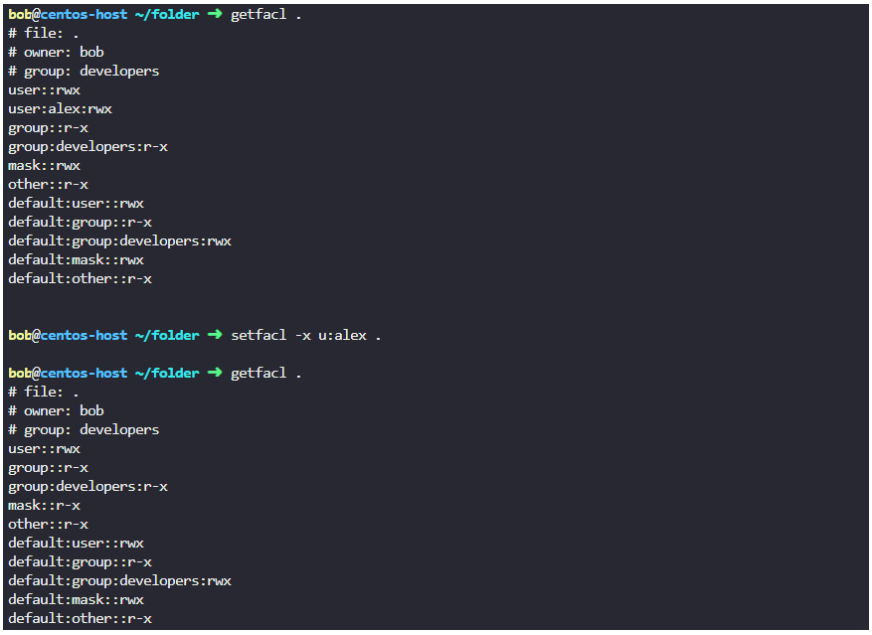
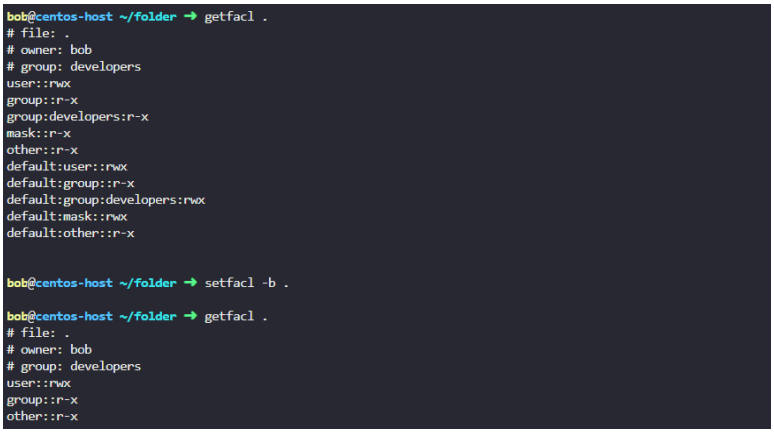
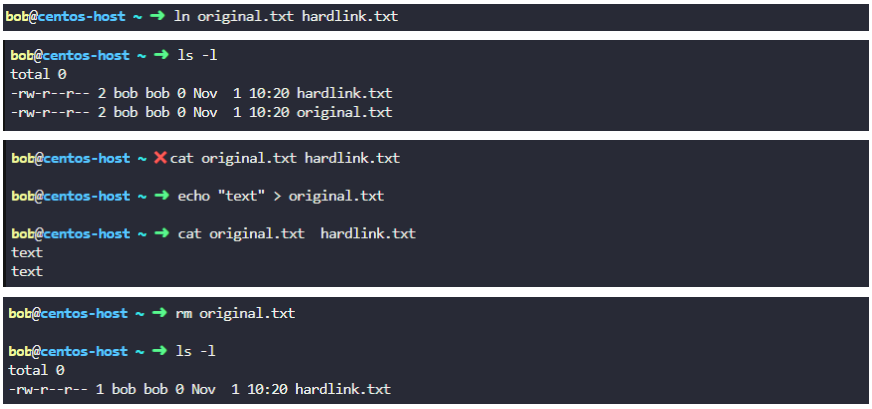
```
bob@centos-host ~/folder ➜ getfacl .
# file: .
# owner: bob
# group: developers
user::rwx
group::r-x
other::r-x
```

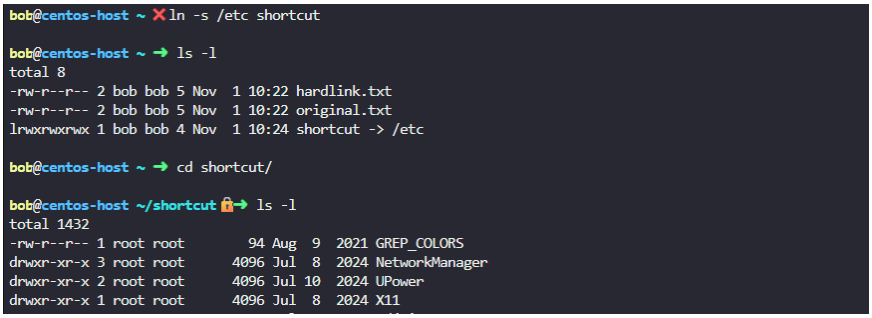
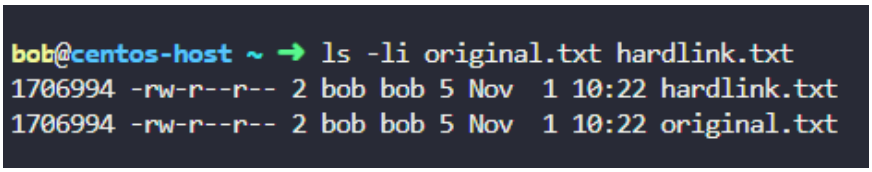
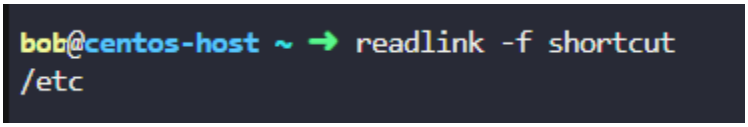
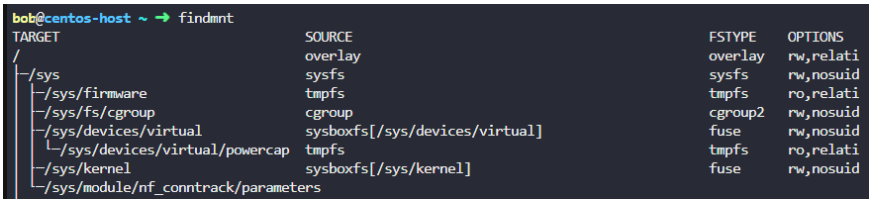
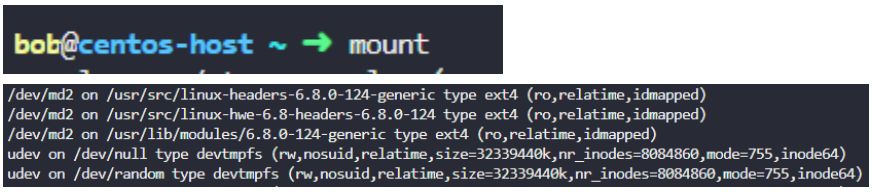
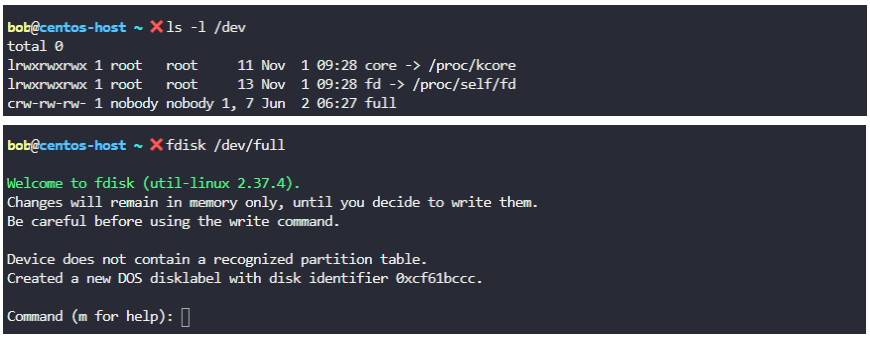
</td>
</tr>
</table>

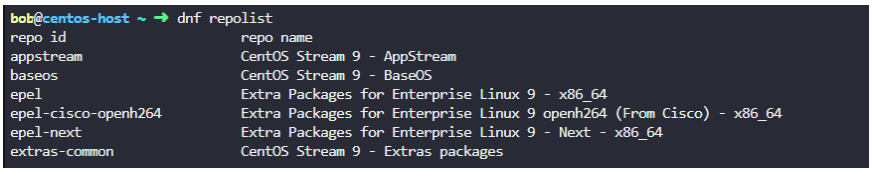
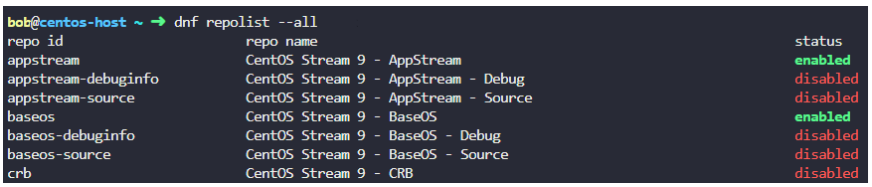
	`bob@centos-host ~/folder ➜ setfacl -m u:alex:rwx .`
	
	`bob@centos-host ~/folder ✗ getfacl .`
	`# file: .`
	`# owner: bob`
	`# group: developers`
	`user::rwx`
	`user:alex:rwx`
	`group::r-x`
	`mask::rwx`
	`other::r-x`

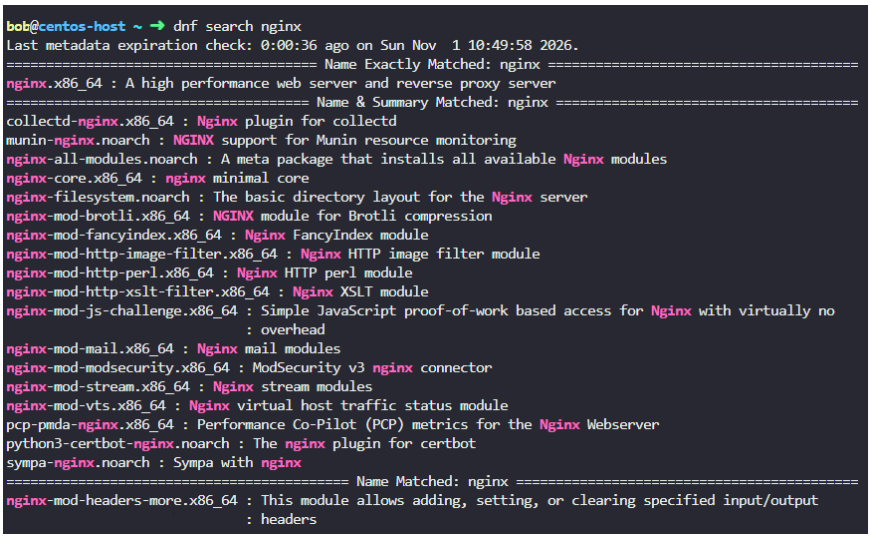
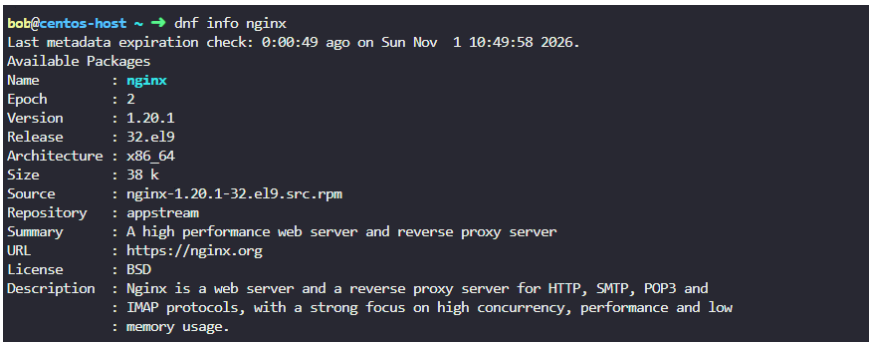
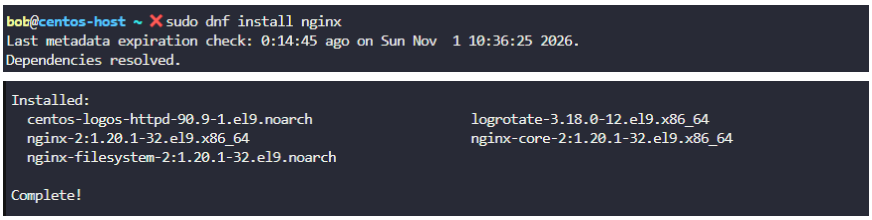
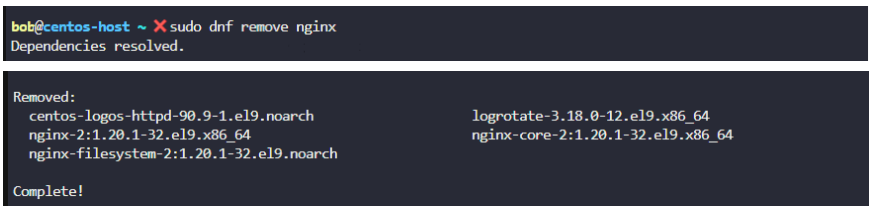
setfacl -m g:developers:rx .	`bob@centos-host ~/folder ➜ getfacl .`
	`# file: .`
	`# owner: bob`
	`# group: developers`
	`user::rwx`
	`user:alex:rwx`
	`group::r-x`
	`mask::rwx`
	`other::r-x`
	
	`bob@centos-host ~/folder ➜ setfacl -m g:developers:rx .`
	
	`bob@centos-host ~/folder ➜ getfacl .`
	`# file: .`
	`# owner: bob`
	`# group: developers`
	`user::rwx`
	`user:alex:rwx`
	`group::r-x`
	`group:developers:r-x`
	`mask::rwx`
	`other::r-x`

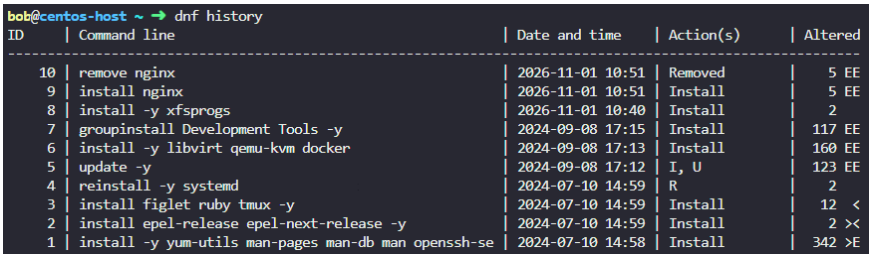
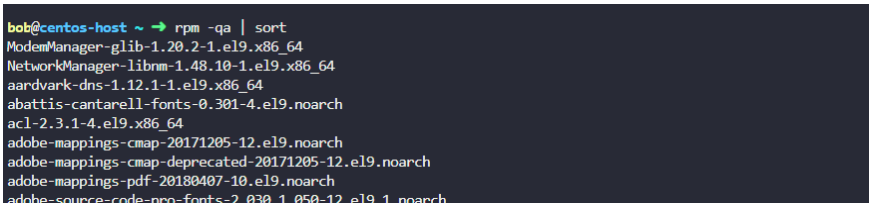
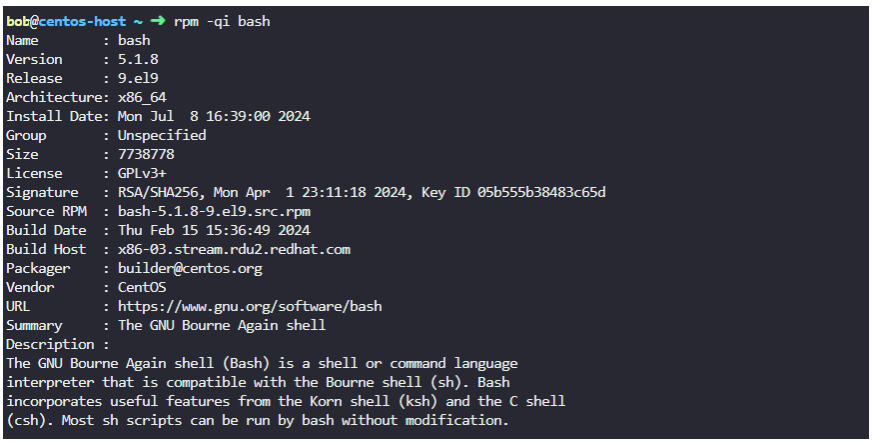
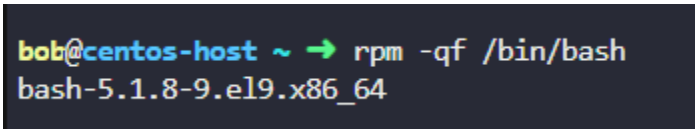
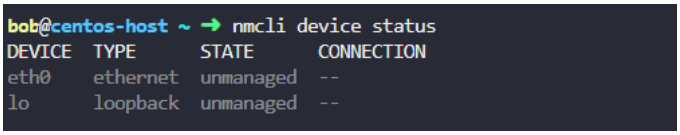
setfacl -m d:g:developers:rwx .	`bob@centos-host ~/folder ➜ getfacl .`
	`# file: .`
	`# owner: bob`
	`# group: developers`
	`user::rwx`
	`user:alex:rwx`
	`group::r-x`
	`group:developers:r-x`
	`mask::rwx`
	`other::r-x`
	
	`bob@centos-host ~/folder ➜ setfacl -m d:g:developers:rwx .`
	
	`bob@centos-host ~/folder ➜ getfacl .`
	`# file: .`
	`# owner: bob`
	`# group: developers`
	`user::rwx`
	`user:alex:rwx`
	`group::r-x`
	`group:developers:r-x`
	`mask::rwx`
	`other::r-x`
	`default:user::rwx`
	`default:group::r-x`
	`default:group:developers:rwx`
	`default:mask::rwx`
	`default:other::r-x`

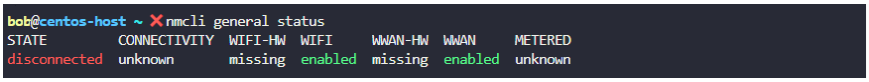
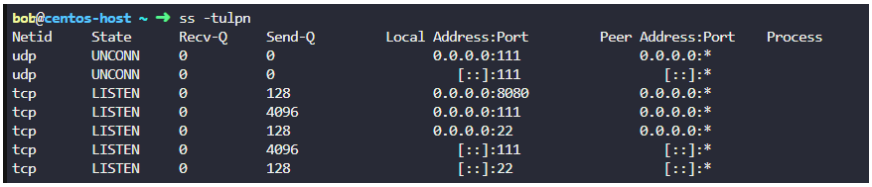
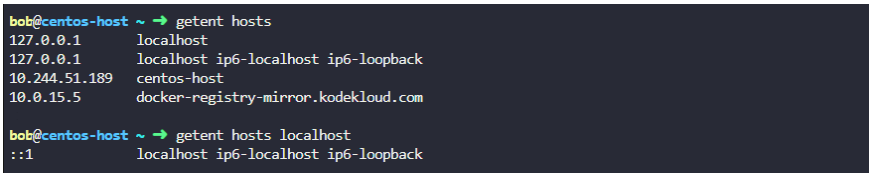
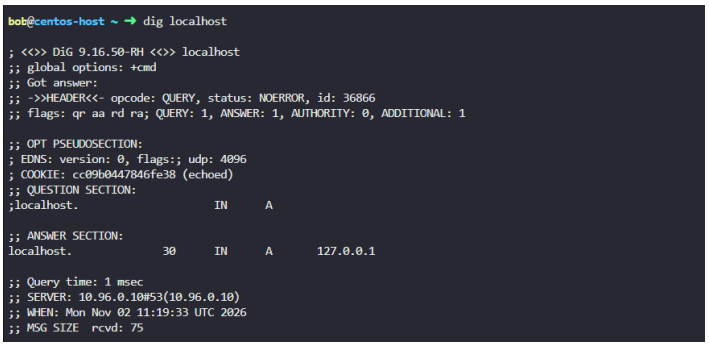
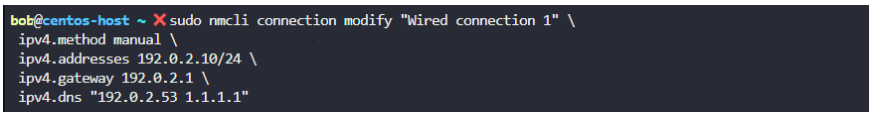
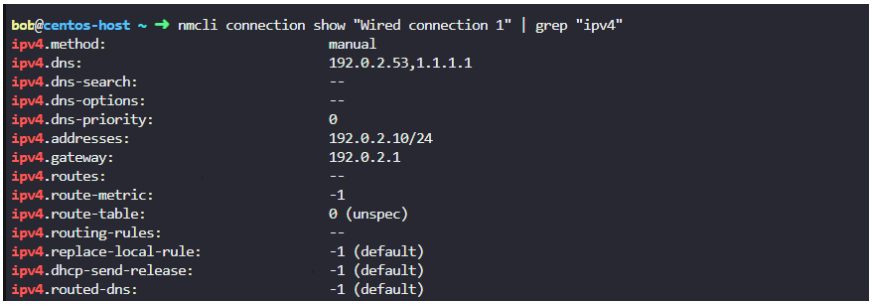
setfacl -x u:alex .	
setfacl -b .	
(Hard Link) Points to same INODE ln original.txt hardlink.txt If you edit original.txt, hardlink.txt syncs. If you delete original.txt, hardlink.txt remains.	

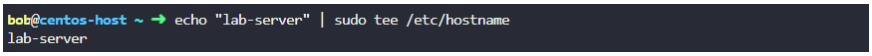
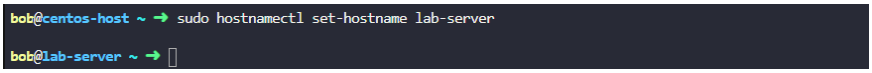
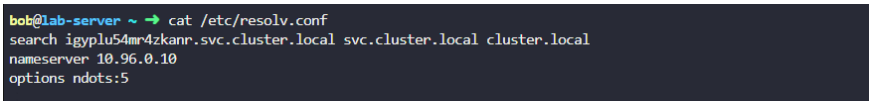
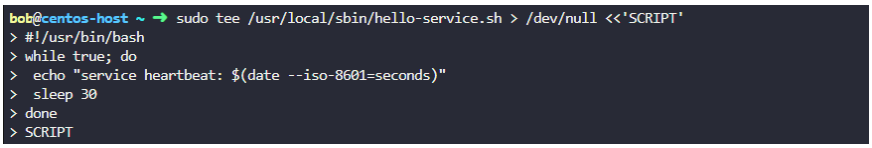
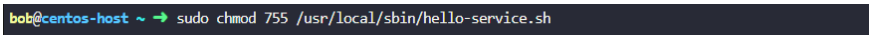
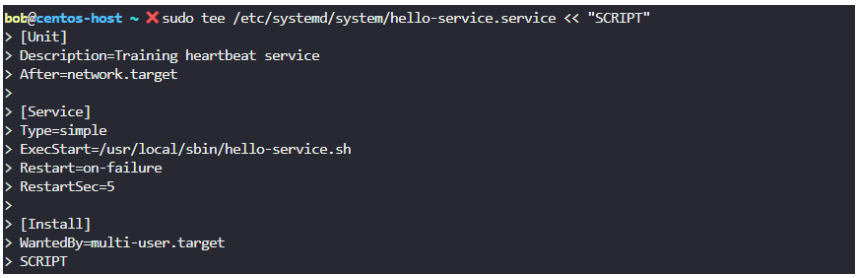
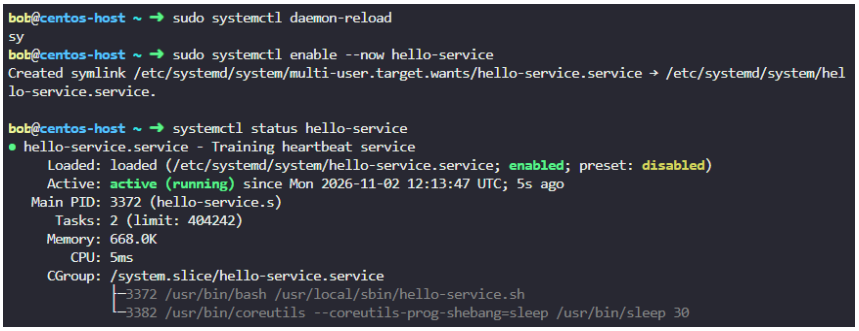
(Symbolic Link) Hyperlink || Shortcut ln -s /var/log/messages messages-link If you delete original file/path, the shortcut is dead	
ls -li original.txt hardlink.txt shows same INODE (location to the storage)	
readlink -f shortcut	
findmnt	
mount	
(Disk Partitioning) fdisk /dev/full	
(Reload Partition Table) partprobe /dev/full	`partprobe` **saves you a reboot** by instantly telling the kernel: *"Hey, the partitions on this disk just changed. Please update your records right now."*

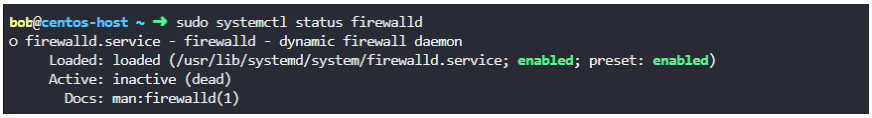
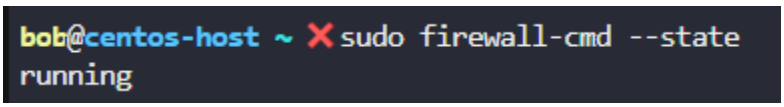
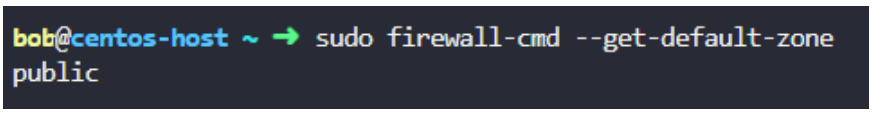
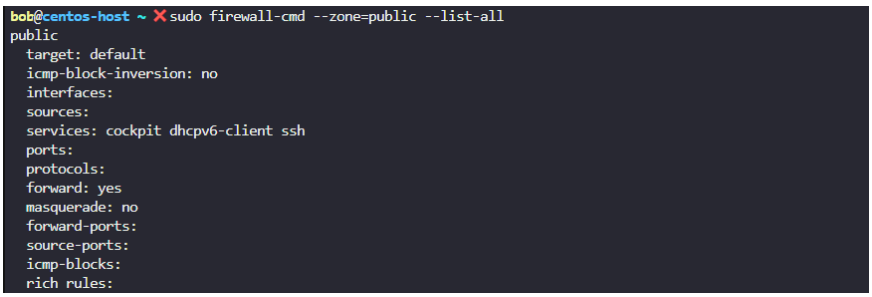
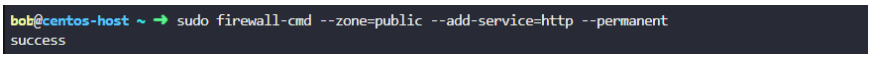
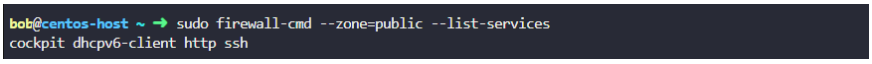
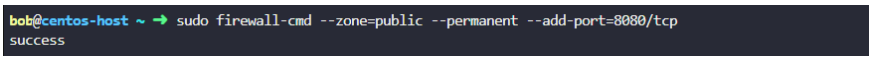
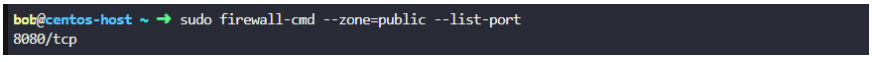
(Make File System - Extensible File System) mkfs.xfs /dev/sda1	format a disk partition with the XFS file system If you then run `sudo blkid` or `sudo lsblk -f` they will suddenly light up and show `FSTYPE="xfs"` along with a unique **UUID** string.
blkid	display attributes of storage devices, primarily their unique UUIDs and file system types
sudoedit	edit system configuration files that require root permissions When you run `sudoedit /etc/fstab`, it doesn't actually open the real file. Instead: 1. It securely makes a **temporary copy** of the system file. 2. It opens that copy using your regular, unprivileged text editor. 3. Once you save and close the editor, `sudoedit` checks if you made changes and **writes them back** to the original system file.
findmnt /tmp	
DO 5.6 TO 5.8	
DNF same as YUM commands dnf repolist	
dnf repolist --all	

dnf search nginx	
dnf info nginx	
dnf install nginx	
dnf remove nginx	
dnf update	updates dnf
dnf check-update	checks if there are updates on dnf
dnf clean all	wipe out cached package data from your system
dnf makecache	downloads and stores the latest metadata for all enabled software repositories.

dnf history	
rpm -qa | sort	
rpm -qi bash	
rpm -ql bash	
rpm -qf bash	
nmcli device status	
nmcli connection show	

nmcli general status	
ss -tulpn	
getent hosts	
dig localhost	
ip neigh	
sudo nmcli connection modify "Wired connection 1" \ ipv4.method manual \ ipv4.addresses 192.0.2.10/24 \ ipv4.gateway 192.0.2.1 \ ipv4.dns "192.0.2.53 1.1.1.1" sudo nmcli connection up "Wired connection 1" nmcli connection show "Wired connection 1" sudo nmcli connection modify "Wired connection 1" ipv4.method auto	 

echo “lab-server” | sudo tee /etc/hostname	
sudo hostnamectl set-hostname lab-server	
cat /etc/resolv.conf	
ssh <user>@<hostname>	ex. ssh tony@stapp01 | ssh steve@stapp02
scp file.txt <user>@<hostname>:/tmp/	USE /tmp/ or ~/ in doing SCP.
systemctl status sshd systemctl is-active sshd systemctl is-enabled sshd systemctl start sshd systemctl stop sshd systemctl restart sshd systemctl reload sshd systemctl enable sshd systemctl enable --now sshd systemctl disable --now sshd	enable --now = enable + start disable --now = disable + stop
SAMPLE SCRIPT	   

systemctl status	
firewall-cmd --state	
firewall-cmd --get-active-zone	
firewall-cmd --get-default-zone	
firewall-cmd --zone=public --list-all	
firewall-cmd --get-services	
firewall-cmd --zone=public --permanent --add-service=http	 
firewall-cmd --zone=public --permanent --add-port=8080/tcp	 

Bash Scripting 1	#!/usr/bin/bash set -Eeuo pipefail read -r -p "Create a file: " file if [[-z "$file"]]; then echo "Don't leave empty" >&2 exit 1 fi if [[-n "$file"]]; then echo "Printing file named $file" touch "$file" echo "File created" exit 1 fi
Bash Scripting 2	#!/usr/bin/bash set -Eeuo pipefail name="Linux Learner" printf "The name is: %s" "$name"
Bash Scripting 3	#!/usr/bin/bash set -Eeuo pipefail source_dir="/etc" backup_dir="/var/backups/etc" mkdir -p "$backup_dir" archive="$backup_dir/etc$ −(date +%F-%H%M%S).tar.gz" if [[! -d "$source_dir"]]; then echo "Source directory does not exist: $source_dir" >&2 exit 1 fi tar -czf "$archive" -C / etc tar -tzf "$archive" >/dev/null sha256sum "$archive" > "archive.sha256" sha256sum -c "$archive.sha256" printf 'Backup created and verified: %s\n' "$archive"
crontab -l	shows crontabs
crontab -e	edit crontab
CRON FORMAT	minute hour day-of-month month day-of-week command

systemctl start crond	start cron
Crontab 1	15 2 * * * /usr/local/sbin/backup-etc.sh >> /var/log/backup-etc.log 2>&1

None

```
# 1. Identify your local IP and network
ip -brief address
ip route

# 2. Discover active hosts on a subnet
nmap -sn 192.168.1.0/24

# 3. Scan a specific host
nmap 192.168.1.10

# 4. Scan specific ports and detect services
nmap -sV -p 22,80,443 192.168.1.10

# 5. Save scan results to a file
nmap -oN scan_results.txt 192.168.1.10

# 6. Combine scanning and saving
nmap -sV -p 22,80,443 -oN results.txt 192.168.1.10
```

Meaning of flags:

Flag	Meaning
`-sn`	Discover hosts without port scanning
`-sV`	Detect service versions
`-p`	Specify port numbers
`-oN`	Save output in normal text format
`/24`	Network prefix representing the subnet

None

```
#!/bin/bash

read -r -p "Enter filename: " file

if [[ -z "$file" ]]; then
    echo "Filename cannot be empty"
    exit 1

elif [[ ! -f "$file" ]]; then
    echo "File does not exist"
    exit 1

elif rm -- "$file"; then
    if [[ ! -e "$file" ]]; then
        echo "File successfully deleted"
    else
        echo "File deletion failed"
        exit 1
    fi

else
    echo "Unable to delete file"
    exit 1
fi
```

Meaning of Bash syntax:

Syntax	Meaning
`#!/bin/bash`	Execute using Bash
`read -r`	Read user input
`-p`	Display an input prompt
`$file`	Access variable value
`-z`	Check if string is empty
`-f`	Check if regular file exists
`!`	NOT / reverse condition
`-e`	Check if path exists
`elif`	Else if
`rm`	Remove file
`--`	End option parsing; protects filenames starting with `-`
`&&`	AND
`||`	OR
`exit 1`	Exit with error
`exit 0`	Exit successfully

LABORATORY

Easy

Lab E1 - System inventory (10 minutes)
Scenario: You have been assigned an unfamiliar CentOS VM. Create a concise system inventory.

Requirements: Identify release, kernel, hostname, current user and groups, IP addresses, default route, filesystems, memory, uptime, and failed systemd units.

Success criteria: Your notes contain command output or concise values for all nine areas. Do not change system configuration.

```
cat /etc/os-release
uname -r
hostname || hostnamectl
cat /etc/passwd
cat /etc/group
ip -brief address
ip route
df -h
free -h || lsmem
uptime
systemctl --failed
```

Lab E2 - User and group creation (12 minutes)
Scenario: A small team needs a shared working directory.

Requirements: Create group ops; create users mika and ravi with home directories and Bash shells; add both to ops without removing existing supplementary groups; create /srv/ops; set group ownership to ops; configure the directory so new files inherit group ops; restrict access to users outside the owner/group.

Success criteria: id shows membership; ls -ld shows SGID and no other access; both users can traverse/write if tested with sudo -u.

```
sudo groupadd ops
sudo useradd -m -s /bin/bash mika
sudo useradd -m -s /bin/bash ravi
sudo usermod -aG ops mika
sudo usermod -aG ops ravi
mkdir -p /srv/ops
sudo chown root:ops /srv/ops
sudo chmod 2770 /srv/ops
```

```
Checking:
id mika
id ravi
ls -ld /srv/ops
sudo -u mika touch /srv/ops/mika_file.txt
sudo -u ravi touch /srv/ops/ravi_file.txt
ls -l /srv/ops
```

Lab E3 - Package query and install (10 minutes)
Scenario: The administrator needs tree to inspect directory structures.

Requirements: Identify the release's package manager; search for tree; install it; verify the owning package and executable path.

Success criteria: tree runs and rpm -qf identifies its package

```
sudo dnf search tree
sudo dnf install -y tree
which tree
sudo rpm -qf $(which tree)
```

Lab E4 - Service state check (10 minutes)
Scenario: A host should have SSH available and configured to start at boot.

Requirements: Inspect sshd; start it if inactive; enable it at boot; verify active state, enablement state, and listening socket.

Success criteria: systemctl is-active sshd, systemctl is-enabled sshd, and socket inspection support the requested state.

```
sudo systemctl sshd enable --now
sudo systemctl is-active sshd
sudo systemctl is-enabled sshd
ss -tulpn | grep “22”
```

Average:
Lab A1 - ACL-based shared workspace (20 minutes)
Scenario: developers own /srv/app. A user named qauser must list and traverse the directory but must not create, rename, or modify entries. If the task also requires reading existing files, apply a separate named-user read ACL to the relevant files. New files created in the directory should inherit group developers.

Requirements: Create or verify the group and user; configure ownership and group inheritance; set base permissions; grant qauser read/traverse ACL access; verify effective ACLs and parent directory traversal.

Success criteria: getfacl shows the named-user entry and group inheritance; an access test as qauser can list/read permitted content but cannot create or modify files.

```
sudo groupadd developers
sudo useradd qauser -m -s /bin/bash
sudo chmod 2770 /srv/app
sudo setfacl -m u:qauser:r-x /srv/app
sudo setfacl -d -m u:qauser:r-x /srv/app
sudo getfacl /srv/app
```

```
To check:
sudo -u qauser ls -la /srv/app
sudo -u qauser touch /srv/app
```

Lab A2 - Network diagnosis and static profile (20 minutes)
Scenario: A lab VM has the wrong IPv4 configuration. The lab network administrator gives you a profile specification.

Requirements: Identify interface and active NetworkManager profile; record the original settings; set the assigned static IP/prefix, gateway, and DNS values from your instructor's isolated lab sheet; reactivate the profile; verify address, route, and DNS resolution.

Success criteria: Interface and route match the provided specification. Do not use the example documentation IPs from this guide unless your isolated network is explicitly configured for them.

```
nmcli connection show --active  // e.g., ens160 or Wired connection 1
nmcli connection show ens3 > ens3config.txt
sudo nmcli connection modify "ens3" ipv4.method manual \
ipv4.addresses "192.168.2.25/24" \
ipv4.gateway "192.168.2.1" \
ipv4.dns "192.168.1.10" && sudo nmcli connection up "ens3"
nmcli device status
```

```
DEVICE  TYPE      STATE      CONNECTION
ens3    ethernet  connected  ens3
lo      loopback  unmanaged  --
```

nmcli connection show

```
NAME    UUID                                  TYPE      DEVICE
ens3    4a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d  ethernet  ens3
```

Lab A3 - Persistent data mount (25 minutes)
Scenario: A newly added 5-10 GB virtual disk must be mounted at /data after reboot.

Requirements: Identify the spare disk; create one partition; format it XFS; mount it at /data; create an fstab entry using the filesystem UUID; validate fstab and mount state.

Success criteria: findmnt --verify reports no configuration error, mount -a succeeds, findmnt /data shows the correct filesystem, and the mount persists after a controlled reboot.

```
lsblk -f                        # spare disk: no partitions, no FSTYPE, no mountpoint (used /dev/vdd)
sudo fdisk /dev/vdd             # n, p, 1, Enter, Enter, w
sudo partprobe /dev/vdd
sudo mkfs.xfs -f /dev/vdd1
sudo mkdir -p /data
echo "UUID=$(sudo blkid -s UUID -o value /dev/vdd1) /data xfs defaults 0 0" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload
sudo umount /data 2>/dev/null
sudo findmnt --verify           # no errors
sudo mount -a                   # silent = success
findmnt /data                   # SOURCE /dev/vdd1, FSTYPE xfs
```

Do this if it went wrong:

```
sudo vi /etc/fstab              // remove the UUID
wipefs -a /dev/vdd
sudo partprobe /dev/vdd
```

Lab A4 - Package, service, and firewall (20 minutes)
Scenario: Install the Apache HTTP server and make the standard HTTP service reachable on the lab VM.

Requirements: Install the correct package; enable and start the service; add the HTTP service to the appropriate firewalld zone permanently; reload firewall rules; verify service state, listener, and firewall rule.

Success criteria: httpd is active/enabled, port 80 is listening, and the active zone allows http. A separate client test is recommended if a second VM exists.

```
sudo dnf install -y httpd firewalld
sudo systemctl enable --now httpd
```

sudo systemctl enable --now firewalld
sudo firewall-cmd --zone=public --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --zone=public --list-services
ss -tulpn | grep “80”
curl -I http://localhost

Hard
Lab D1 - Diagnose a broken web service (35 minutes)
Scenario: Apache is installed, but the page cannot be retrieved from a second VM. The service may have a configuration, port, firewall, filesystem-permission, or SELinux issue.

Requirements: Diagnose without disabling SELinux or firewalld; identify the actual cause; correct only the necessary settings; verify local HTTP response, listening address/port, firewall rule in the active zone, and SELinux labels for content; document the original fault and proof of repair.

Success criteria: A client VM can retrieve the required page over HTTP, the service is enabled, and security controls remain enabled.

systemctl status httpd
journalctl -u httpd -b --no-pager
sudo httpd -t
sudo ss -lntp
curl -v http://127.0.0.1/
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --zone=<active-zone> --list-all
getenforce
ls -Zd /var/www/html

Lab D3 - Custom systemd service (30 minutes)
Scenario: A script at /usr/local/sbin/heartbeat.sh must run continuously as a systemd service and restart on failure.

Requirements: Create a safe script; set executable permissions; create a valid service unit; reload systemd; enable and start it; inspect logs; stop and start it; verify it is enabled and active.

Success criteria: Unit is active/enabled, logs show repeated output, and systemd reports expected restart behavior. Do not use an infinite loop that consumes CPU; include a sensible sleep.

sudo tee /usr/local/sbin/heartbeat.sh << SCRIPT
#!/usr/bin/bash
while true; do

```
 echo "heartbeat $(date --iso-8601=seconds)"
 sleep 30
done
SCRIPT

sudo chmod 755 /usr/local/sbin/heartbeat.sh

sudo tee /etc/systemd/system/heartbeat.service << SCRIPT
[Unit]
Description=Training heartbeat
After=network.target
[Service]
Type=simple
ExecStart=/usr/local/sbin/heartbeat.sh
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
SCRIPT

sudo systemctl daemon-reload
sudo systemctl enable --now heartbeat.service
systemctl is-active heartbeat.service
systemctl is-enabled heartbeat.service
journalctl -u heartbeat.service -n 20 --no-pager
```